

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS



MicroTorch

A From-Scratch C++ Automatic Differentiation Engine

A Project Report

Submitted in partial fulfillment of the requirements of the subject

Object-Oriented Programming (ENCT 151)

Submitted to:

Department of Electronics and Computer Engineering

Submitted by:

Utsab Raj Bhattarai (082BCT095)

Sonik Bhusal (082BCT085)

Shubham Pokhrel (082BCT84)

Submission Date: 20 August, 2026

Acknowledgment

We would like to express our sincere gratitude to our subject teacher and the Department of Electronics and Computer Engineering, Pulchowk Campus, for providing us the opportunity to undertake this project as part of the Object-Oriented Programming course.

This project, MicroTorch, challenged us to build a deep-learning framework from first principles in C++, and in doing so it deepened our understanding of both object-oriented design and the mathematics of automatic differentiation. We are grateful for the guidance and encouragement that made this possible.

We also thank our peers and teammates for the many discussions that shaped the design decisions documented in this report, and our families for their constant support throughout the development of this work.

Abstract

MicroTorch is a reverse-mode automatic differentiation (autograd) engine implemented entirely from scratch in C++, using only the Eigen library for matrix arithmetic and no machine-learning or application frameworks. The project set out to demonstrate that a general-purpose deep-learning framework can be constructed and rigorously verified from primitive operations, and that such an engine is genuinely general rather than tailored to a single model.

To establish generality, two fundamentally different classes of model were built on the same operator set. The first is a Graph Neural Network (GNN) applied to anti-money-laundering (AML) node classification on the Elliptic Bitcoin dataset, a discriminative task. The second is a denoising diffusion probabilistic model (DDPM) that generates two-dimensional shapes from pure noise, a generative task. Both models share the same tensors, operations, optimizers, and backward pass, with no engine-side special-casing for either.

Every operation in the engine was verified during development by comparing its analytically derived gradient against a numerical finite-difference approximation, ensuring correctness of the backward pass to within a tolerance of $1e-6$ or better. The project additionally contributes two interactive desktop visualizations built with the raylib graphics library: an AML analyst dashboard for the GNN and an animated denoising viewer for the diffusion model.

Beyond the engineering, the project produced two honest empirical findings. A controlled ablation showed that on the feature-rich Elliptic dataset a plain multilayer perceptron matched or exceeded the graph model, an over-smoothing effect consistent with published literature. The diffusion demo cleanly generated well-separated distributions while blurring tightly-wound manifolds, reflecting the representational limits of a small denoiser. These results are reported as legitimate scientific outcomes, reinforcing that the engine, not any single model result, is the product of this work.

Table of Contents

Acknowledgment	2
Abstract	3
Table of Contents	4
1. Objectives	6
2. Introduction	7
3. Application	8
3.1 Anti-Money-Laundering Detection	8
3.2 Generative Modeling	8
3.3 Educational and Research Value	8
4. Literature Survey	9
4.1 Automatic Differentiation	9
4.2 Graph Neural Networks	9
4.3 Diffusion Models	9
4.4 Object-Oriented Design	9
5. Existing System	10
6. Methodology	11
6.1 The Autograd Core	11
6.2 Operations	11
6.3 Verification by Gradient Checking	11
6.4 Models and Optimizers	12
7. Implementation	13
7.1 System Architecture	13
7.1.1 Block Diagram	13
7.2 The Engine	13
7.3 The Graph Neural Network	14
7.4 The Diffusion Model	14
7.5 The Visualizations	15
8. Results	16
8.1 Engine Verification	16
8.2 GNN Classification Performance	16
8.3 The GNN-versus-MLP Ablation	16
8.4 Diffusion Generation	17
9. Problems Faced and Solutions	18
9.1 Memory Cycles in the Backward Closure	18
9.2 Silent, Compile-Clean Bugs	18
9.3 Oscillating Diffusion Loss	18
9.4 The Diffusion Quality Plateau	18
9.5 Build-System Linker Errors	18
10. Limitations and Future Enhancements	20
10.1 Limitations	20
10.2 Future Enhancements	20
11. Conclusion and Recommendations	21

11.1 Conclusion	21
11.2 Recommendations	21
12. References	22

1. Objectives

The objectives of this project are:

- To design and implement a reverse-mode automatic differentiation engine in C++ from first principles, depending only on a matrix-arithmetic library and no machine-learning frameworks.
- To implement a complete set of differentiable operations, each with a hand-derived backward pass, and to verify their correctness numerically.
- To demonstrate the generality of the engine by building two fundamentally different models on it: a discriminative Graph Neural Network and a generative diffusion model.
- To apply object-oriented programming principles, encapsulation, inheritance, polymorphism, abstraction, and composition, meaningfully throughout the design rather than as decoration.
- To produce interactive visualizations that make the engine's results legible and demonstrable.
- To evaluate the models rigorously and report findings honestly, including results that do not favor the more complex model.

2. Introduction

Modern deep-learning frameworks such as PyTorch and TensorFlow are built around a single central idea: automatic differentiation. Rather than requiring the programmer to derive and code gradients by hand, these frameworks record the operations performed during a forward computation and then automatically compute the gradients of a scalar loss with respect to every parameter by applying the chain rule in reverse. This mechanism, known as reverse-mode automatic differentiation or backpropagation, is what makes training neural networks tractable.

MicroTorch reconstructs this mechanism from scratch. At its core is a Tensor type that acts as a node in a computation graph. Each tensor stores its value, an accumulator for its gradient, references to the tensors it was computed from (its parents), and a closure describing how to propagate gradient to those parents. As operations execute during the forward pass, they build this graph implicitly. When the backward pass is invoked on a scalar loss, the engine performs a topological traversal of the graph and fires each node's stored gradient rule in reverse order, so that every parameter ends with the correct gradient of the loss with respect to it.

Building such an engine is a demanding exercise in both mathematics and software engineering. Each operation requires a correctly derived gradient; the memory model must keep the graph alive without leaking; and the whole system must be organized cleanly enough that entirely different model architectures can be expressed on it. This report documents that construction, the two demonstrator models built upon it, the visualizations that showcase them, and the empirical findings that emerged.

3. Application

The engine and its demonstrator models have direct real-world applications:

3.1 Anti-Money-Laundering Detection

The Graph Neural Network demonstrator addresses financial crime detection. Money laundering is inherently a network phenomenon: illicit funds move through chains of transactions between accounts. Modeling transactions as a graph and classifying nodes as licit or illicit is a realistic formulation of the problem, and the accompanying analyst dashboard illustrates how such a model could support investigators by ranking suspicious accounts and visualizing their transaction neighborhoods.

3.2 Generative Modeling

The diffusion demonstrator represents the family of generative models now widely used for image, audio, and data synthesis. Although this project generates simple two-dimensional shapes, the same mechanism, learning to reverse a gradual noising process, underlies state-of-the-art generative systems. Building it from scratch provides an educational, transparent view of how such models work.

3.3 Educational and Research Value

Most fundamentally, the project is an educational and research artifact. By exposing every internal mechanism, the tensor graph, the backward pass, the operations, and the optimizers, without the abstraction of an existing framework, it serves as a transparent reference for understanding how deep-learning systems operate internally.

4. Literature Survey

The design of MicroTorch draws on several established bodies of work.

4.1 Automatic Differentiation

Reverse-mode automatic differentiation is the foundation of modern neural-network training. The define-by-run (dynamic graph) approach popularized by autograd and later by PyTorch, in which the computation graph is built implicitly as operations execute, directly informed MicroTorch's design, where each operation records its own backward rule as a closure at the moment it runs.

4.2 Graph Neural Networks

Graph Neural Networks extend neural networks to graph-structured data by aggregating information across a node's neighbors. The Graph Convolutional Network (GCN) formulation, which uses a symmetrically normalized adjacency matrix to average neighbor features, is the basis of the aggregation step implemented in this project. The Elliptic Bitcoin dataset used here is an established benchmark for graph-based financial fraud detection.

4.3 Diffusion Models

Denoising Diffusion Probabilistic Models (DDPMs) generate data by learning to reverse a fixed process that gradually adds Gaussian noise. The predict-the-noise training objective, the linear variance schedule, and the ancestral sampling procedure implemented in this project follow the standard DDPM formulation. The sinusoidal time-step embedding used to condition the denoiser is adapted from the positional-encoding technique introduced for Transformer models.

4.4 Object-Oriented Design

The software design follows widely accepted C++ guidance, in particular the principle that encapsulation exists to protect invariants and that composition should be preferred over inheritance where an is-a relationship does not genuinely hold. These principles directly shaped the decision to expose plain data aggregates while encapsulating stateful components such as models and optimizers.

5. Existing System

Established deep-learning frameworks, most prominently PyTorch and TensorFlow, already provide mature, highly optimized automatic differentiation and a vast library of pre-built layers, optimizers, and models. These systems are the industry standard and are vastly more capable and performant than any student project could be.

However, these existing systems are, from an educational standpoint, opaque. Their autograd engines are implemented in optimized C++ and CUDA beneath a Python interface, and a typical user never sees how a gradient is actually computed, how the computation graph is represented, or how memory is managed during the backward pass. For the purpose of understanding, and for the constraints of an object-oriented programming course that prohibits building on machine-learning frameworks, these systems are unsuitable as a foundation.

MicroTorch therefore does not attempt to compete with these systems on capability or speed. Instead it reconstructs their central mechanism transparently and from scratch, so that every step is visible and every design decision is deliberate. Where existing systems abstract the engine away, MicroTorch makes the engine the explicit subject of study. The only external dependency is the Eigen library, used purely for matrix arithmetic, which plays the same role that low-level linear-algebra libraries play beneath the existing frameworks.

6. Methodology

The project was developed in layers, each verified before the next was built.

6.1 The Autograd Core

The foundation is the Tensor type, a node in a dynamically constructed computation graph. Every tensor holds its value (data), a same-shaped gradient accumulator (grad), a list of parent tensors it was computed from, and a backward closure. Tensors are held through shared pointers so that the graph remains alive for as long as any downstream tensor references it.

The backward pass is implemented as a topological traversal. Starting from the scalar loss, a depth-first search builds a list of all ancestor nodes in an order where every node appears after its parents. The loss gradient is seeded to one (since the derivative of the loss with respect to itself is one), and the list is then walked in reverse, firing each node's backward closure. Because gradients accumulate additively, a tensor used along multiple paths correctly receives the sum of all contributions, and a visited set ensures each node is processed exactly once.

6.2 Operations

Each differentiable operation follows an identical template: it validates input shapes, computes the forward result, creates the output tensor, records the inputs as parents, and installs a backward closure that pushes gradient to those inputs. The closures capture raw pointers to the tensors rather than shared pointers; because the closure is stored inside the output tensor, capturing shared pointers would create an ownership cycle and leak memory, while the parent references already guarantee the tensors stay alive.

6.3 Verification by Gradient Checking

Correctness of every operation's backward pass was established by gradient checking. For each operation, the analytically computed gradient was compared against a numerical estimate obtained by central finite differences, perturbing each input slightly and measuring the resulting change in the loss. Because the finite-difference estimate depends only on the definition of a derivative and not on

any hand-derived formula, it independently confirms that both the implementation and the underlying mathematics are correct. This verification was run as a separate build-time step and is not part of the engine itself.

6.4 Models and Optimizers

On top of the operations, two model classes were built, along with SGD and Adam optimizers. To demonstrate inheritance meaningfully, the optimizers share an abstract `Optimizer` base class that provides the common gradient-clearing behavior and declares the parameter-update step as a pure virtual method, which each concrete optimizer implements according to its own rule.

7. Implementation

7.1 System Architecture

The system is organized into three layers with strictly one-directional dependencies. The engine, compiled as a single static library, depends only on Eigen. The demonstrator applications depend on the engine. The graphical interfaces depend on neither the engine nor the applications; they are decoupled entirely through the filesystem, reading comma-separated-value artifacts that the applications produce. This decoupling makes concrete the guiding principle that the engine, rather than any glue code, does the real work.

7.1.1 Block Diagram

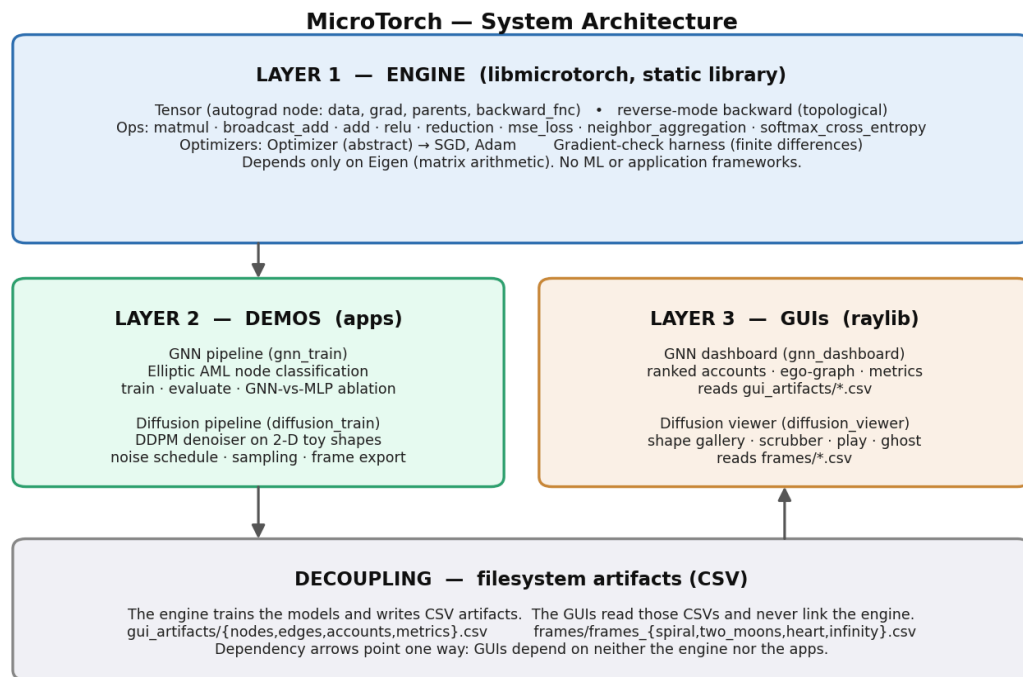


Figure 1: MicroTorch three-layer system architecture.

7.2 The Engine

The engine comprises the Tensor type and its backward pass, a set of eight core operations (matrix multiplication, broadcast addition, element-wise addition, ReLU activation, sum and mean reductions, mean-squared-error loss, sparse

neighbor aggregation, and a fused softmax cross-entropy), the SGD and Adam optimizers under a shared abstract base class, and a gradient-checking harness.

A representative operation is matrix multiplication. Its forward pass computes the product of two matrices; its backward pass propagates gradient to each operand by multiplying the output gradient with the appropriately transposed other operand, an arrangement dictated by the requirement that each gradient share the shape of its corresponding input. The fused softmax cross-entropy operation is notable in that combining the two functions analytically collapses the backward pass to a single subtraction of the target from the predicted probabilities, avoiding an expensive and numerically delicate Jacobian; it is additionally overloaded to support both a fully-labeled case and a masked, semi-supervised case.

7.3 The Graph Neural Network

The GNN is a two-layer graph convolutional network followed by a linear classifier. Each layer aggregates a node's features with those of its neighbors (via multiplication by a symmetrically normalized sparse adjacency matrix), applies a learned linear transformation and bias, and passes the result through a ReLU nonlinearity. The weight matrices are shaped only by feature dimensions and never by the number of nodes, reflecting that the network learns a transformation of features applied uniformly to every node. A single boolean flag disables the neighbor aggregation, converting the model into a plain multilayer perceptron and enabling a controlled ablation.

7.4 The Diffusion Model

The diffusion demonstrator learns to reverse a fixed noising process. A noise schedule defines how much of the original signal survives at each of a thousand timesteps. During training, a clean data point is noised to a random timestep and the denoiser, a small multilayer perceptron, is trained to predict the exact noise that was added, using a mean-squared-error loss. The timestep is provided to the network as a bank of sinusoidal features so that the model can distinguish noise levels. After training, generation proceeds by starting from pure noise and iteratively applying the learned denoiser from the final timestep down to zero, recording the intermediate states as animation frames.

7.5 The Visualizations

Two desktop applications were built with the raylib graphics library. The GNN dashboard presents a ranked list of accounts by predicted risk, a metrics panel, and an interactive ego-graph that, on selecting an account, draws its transaction neighborhood with directional arrows and colors nodes by predicted class. The diffusion viewer presents a gallery of trained shapes; selecting one animates the denoising process from noise to shape, with a draggable timeline, a play control, and a faint overlay of the target shape. Both interfaces read only the CSV artifacts produced by the engine and do not link against it.

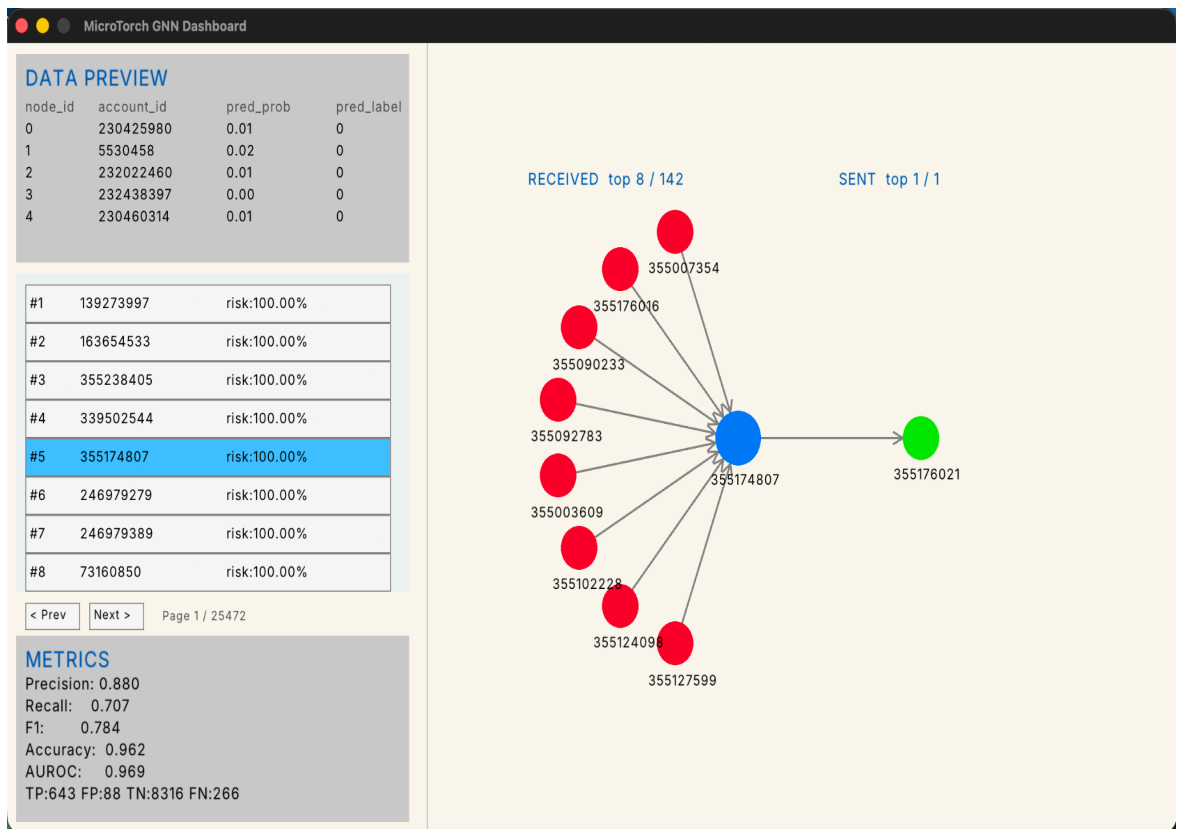


Figure 2: The GNN analyst dashboard, showing the ranked-accounts list, metrics panel, and the interactive transaction ego-graph of a selected account.

8. Results

8.1 Engine Verification

Every operation passed its gradient check, with the maximum discrepancy between analytic and numerical gradients falling below $1e-6$, and the full multi-layer models verifying end-to-end to approximately $1e-10$. This confirms that the backward pass is mathematically correct throughout the engine.

8.2 GNN Classification Performance

On the Elliptic Bitcoin dataset, the graph model achieved strong classification performance on held-out test data. Representative results from a single run are shown below.

Metric	Value
Precision	0.90
Recall	0.77
F1 Score	0.83
Accuracy	0.97
AUROC	0.98

The Area Under the Receiver Operating Characteristic curve (AUROC), computed via the Mann-Whitney rank-sum statistic and verified against a brute-force pair-counting implementation, indicates that the model ranks a randomly chosen illicit node above a randomly chosen licit node roughly 98 percent of the time.

8.3 The GNN-versus-MLP Ablation

The central scientific experiment compared the full graph model against the same architecture with neighbor aggregation disabled (a plain multilayer perceptron), across multiple random seeds, with all other factors held constant. The finding was that the multilayer perceptron consistently matched or exceeded the graph model, most notably in recall.

Configuration	Mean AUROC	Mean Recall
GNN (graph on)	0.970	0.70
MLP (graph off)	0.978	0.80

Because the features already encode neighborhood information, the additional averaging performed by the graph layers dilutes rather than enriches the signal, an over-smoothing effect that is well documented for this dataset. The result is reported honestly as a legitimate empirical finding; the value of the project lies in the engine's ability to express and rigorously compare both models, not in which model wins.

8.4 Diffusion Generation

The diffusion model successfully generated recognizable versions of several two-dimensional distributions, each emerging from pure noise through the learned reverse process. The progression below shows the reverse diffusion of a spiral: the sample begins as pure Gaussian noise, gradually organizes as the denoiser removes noise, and resolves into a recognizable single-arm spiral.

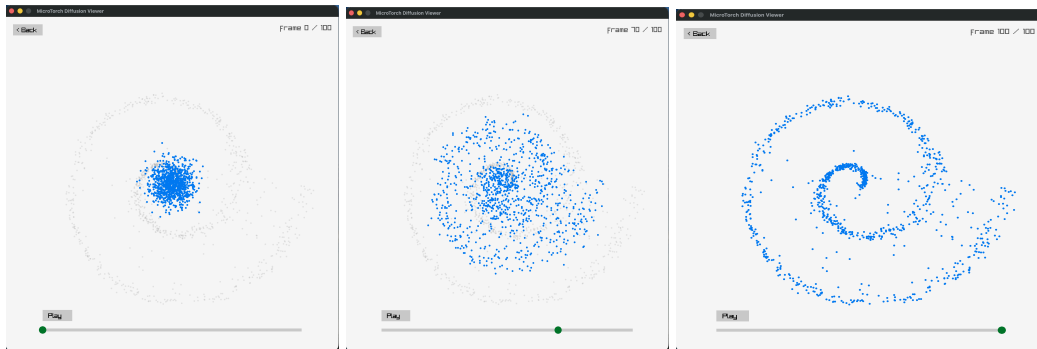


Figure 3: The reverse diffusion process. Starting from pure Gaussian noise (left), the learned denoiser progressively removes noise (centre) a recognizable spiral (right).

Distributions whose structure is well separated in space generated cleanly, whereas tightly-wound or high-frequency structures (a tight double spiral, concentric rings, a checkerboard) were only partially resolved. This reflects the smoothness limitation of a small denoiser, which cannot produce different outputs for nearly identical inputs, and is itself a reportable characteristic of the method rather than a defect of the engine.

9. Problems Faced and Solutions

9.1 Memory Cycles in the Backward Closure

Storing the backward closure inside the output tensor while capturing shared pointers to the inputs created a reference cycle: the tensor effectively owned itself, so its reference count could never reach zero and it leaked. The solution was to capture raw pointers within the closure, allowing it to read and write the tensors' gradients without participating in ownership, since the parent list already keeps all tensors alive for the lifetime of the graph.

9.2 Silent, Compile-Clean Bugs

A recurring class of difficulty was bugs that compiled and ran without error but produced incorrect results: integer division where floating-point was intended, uninitialized gradient accumulators holding garbage, assignment written where subtraction was meant, and integer overflow from multiplying before casting to double. These were caught primarily by the gradient-checking harness, which flagged incorrect derivatives that would otherwise have passed unnoticed.

9.3 Oscillating Diffusion Loss

Early diffusion training exhibited a loss that oscillated rather than converging, because each training step used a single random timestep, producing highly variable gradients. Averaging the loss over several random timesteps per step stabilized training and allowed the loss to fall smoothly.

9.4 The Diffusion Quality Plateau

Even after stabilization, diffusion quality plateaued and generated only blurry shapes. The cause was that the timestep was supplied to the network as a single scalar, too weak a signal for the network to distinguish noise levels. Replacing it with a sinusoidal embedding, encoding the timestep as a bank of sine and cosine features at geometrically spaced frequencies, gave the network a rich per-timestep signal and broke the plateau, enabling recognizable shapes to form.

9.5 Build-System Linker Errors

New source files repeatedly produced undefined-symbol linker errors. The cause was that a newly written implementation file had not been added to the build configuration, so although its declaration was visible, its compiled code did not exist for the linker to find. The solution, applied consistently thereafter, was to register each new source file in the build configuration and reconfigure the project.

10. Limitations and Future Enhancements

10.1 Limitations

- The engine is not performance-optimized to the level of production frameworks; it is designed for clarity and correctness rather than speed, and relies on unoptimized dense matrix operations for most computations.
- The operation set, while sufficient for the two demonstrators, is smaller than that of a full framework; convolution, attention, and many other operations are not implemented.
- The diffusion denoiser is a small multilayer perceptron and cannot capture tightly-wound or high-frequency distributions cleanly, as reflected in the results.
- The graph model implements a single aggregation scheme (symmetric-normalized mean); more expressive schemes such as attention-based aggregation were not explored.

10.2 Future Enhancements

- Extending the operation library to support convolutional and attention-based layers, broadening the range of expressible models.
- Introducing performance optimizations such as operation fusion and more efficient memory reuse in the backward pass.
- Exploring more expressive graph-aggregation schemes and deeper denoiser architectures to improve generative quality.
- Adding a batching abstraction so that training can process subsets of data per step for larger datasets.

11. Conclusion and Recommendations

11.1 Conclusion

This project successfully implemented a reverse-mode automatic differentiation engine from first principles in C++, depending only on a matrix-arithmetic library. The correctness of every operation was verified numerically, and the engine's generality was demonstrated by building two fundamentally different models upon it, a discriminative graph neural network and a generative diffusion model, that share the same operations, optimizers, and backward pass with no engine-side special-casing. Two interactive visualizations make the engine's results tangible.

The project also demonstrates object-oriented design applied by principle rather than by rote: encapsulation is used where invariants must be protected (models and optimizers) and deliberately avoided for plain data aggregates (the tensor node), inheritance is used exactly where an is-a relationship genuinely holds (the optimizer hierarchy), polymorphism appears in three forms (function overloading, virtual dispatch, and closure-based type erasure), and composition forms the structural backbone of the entire system. The empirical findings were reported honestly, reinforcing that the engine is the true product of this work.

11.2 Recommendations

For those undertaking similar work, we recommend building and verifying each operation in isolation before assembling models, as the gradient-checking discipline catches an entire class of silent bugs that would otherwise be extremely difficult to locate. We further recommend treating results that do not favor the more complex model as legitimate findings to be understood and reported, rather than as failures to be hidden, as the scientific value of a controlled comparison is independent of its outcome.

12. References

- [1] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems, 2019.
- [2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations, 2017.
- [3] M. Weber et al., "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," KDD Workshop on Anomaly Detection in Finance, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," Advances in Neural Information Processing Systems, 2020.
- [5] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, 2017.
- [6] G. Guennebaud, B. Jacob et al., "Eigen v3," 2010. [Online]. Available: <https://eigen.tuxfamily.org>.
- [7] B. Stroustrup and H. Sutter, "C++ Core Guidelines." [Online]. Available: <https://isocpp.github.io/CppCoreGuidelines>.
- [8] A. Karpathy, "Neural Networks: Zero to Hero." [Online educational lecture series].